



Security Review For Meteora



Collaborative Audit Prepared For:
Lead Security Expert(s):

Meteora
Oxeix

Date Audited:

g
May 6 - May 9, 2026

Introduction

Referral Staking is a program that lets users stake SPL tokens into a protocol-managed vault and earn rewards computed offchain by the pool owner.

Scope

Repository: [MeteoraAg/referral-staking](https://github.com/MeteoraAg/referral-staking)

Audited Commit: [29bfc910ac23174edc5672e9b97f7f81aab6d097](https://github.com/MeteoraAg/referral-staking/commit/29bfc910ac23174edc5672e9b97f7f81aab6d097)

Final Commit: [aef3a35324eae5f29d2f0bdd91ff4f35bc028190](https://github.com/MeteoraAg/referral-staking/commit/aef3a35324eae5f29d2f0bdd91ff4f35bc028190)

Files:

- `programs/referral/src/access_control.rs`
- `programs/referral/src/constants.rs`
- `programs/referral/src/const_pda.rs`
- `programs/referral/src/instructions/mod.rs`
- `programs/referral/src/instructions/permissioned/ix_transfer_owner.rs`
- `programs/referral/src/instructions/permissioned/ix_update_config.rs`
- `programs/referral/src/instructions/permissioned/ix_update_penalty_fee_receiver.rs`
- `programs/referral/src/instructions/permissioned/mod.rs`
- `programs/referral/src/instructions/permissionless/ix_cancel_unstake.rs`
- `programs/referral/src/instructions/permissionless/ix_claim_penalty_fee.rs`
- `programs/referral/src/instructions/permissionless/ix_close_escrow.rs`
- `programs/referral/src/instructions/permissionless/ix_immediate_unstake.rs`
- `programs/referral/src/instructions/permissionless/ix_initialize_escrow.rs`
- `programs/referral/src/instructions/permissionless/ix_initialize_staking.rs`
- `programs/referral/src/instructions/permissionless/ix_stake.rs`
- `programs/referral/src/instructions/permissionless/ix_unstake.rs`
- `programs/referral/src/instructions/permissionless/ix_withdraw.rs`
- `programs/referral/src/instructions/permissionless/mod.rs`
- `programs/referral/src/lib.rs`
- `programs/referral/src/math/fee.rs`
- `programs/referral/src/math/mod.rs`

- `programs/referral/src/math/safe_math.rs`
- `programs/referral/src/states/escrow.rs`
- `programs/referral/src/states/mod.rs`
- `programs/referral/src/states/staking.rs`
- `programs/referral/src/states/unstake.rs`
- `programs/referral/src/utils/mod.rs`
- `programs/referral/src/utils/token.rs`

Final Commit Hash

ae3a35324eae5f29d2f0bdd91ff4f35bc028190

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Events record token-account owners, not the actual receiving token accounts [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/issues/29>

Summary

Token transfer events emit the owner/authority of the recipient token account instead of the actual token account that received the tokens.

Vulnerability Detail

In SPL Token, tokens are held by token accounts, while each token account has an `owner` authority. The program transfers tokens into recipient token accounts such as `owner_token_vault` and `fee_receiver_token_account`.

However, the emitted events record the token account's `owner` field instead of the recipient token account address.

For example:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissionless/ix_immediate_unstake.rs#L118

```
let receiver = ctx.accounts.owner_token_vault.owner;
```

This records the authority of `owner_token_vault`, not `'owner_token_vault.key()`.

The event comments describe the field as the account that receives the token, but the emitted value is not the actual receiving token account. This mismatch can cause incorrect off-chain accounting and indexing:

<https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/events.rs#L113-114>

```
/// The account that receives the unstake token
pub receiver: Pubkey,
```

<https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/events.rs#L183-184>

```
/// The account that receives the lamports from closing the unstake account
pub rent_receiver: Pubkey,
```

Impact

This can mislead indexers, analytics systems, accounting tools, and downstream integrations that rely on events to track token flows.

Tool Used

Manual Review

Recommendation

Emit both the token account owner and the actual receiving token account address.

For unstake and withdraw events, use fields such as:

```
receiver_owner: Pubkey,  
receiver_token_account: Pubkey,
```

Discussion

codewithgun

Fixed in <https://github.com/MeteoraAg/referral-staking/pull/14>, we will emit only receiver_token_account for now.

Issue L-2: Accrued penalty fees can be redirected to a new fee receiver [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/issues/30>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Accrued penalty fees are stored as a single global counter in the staking account and are not tied to the fee receiver that was configured when the fees were generated.

Vulnerability Detail

When users perform an immediate unstake, the penalty fee is added to the staking account's global `penalty_fee_amount` field:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissionless/ix_immediate_unstake.rs#L127

```
self.penalty_fee_amount = self.penalty_fee_amount.safe_add(penalty_fee_amount)?;
```

This amount is not tracked per fee receiver. The program only stores one accumulated penalty fee balance for the whole staking pool.

The staking owner can later update the configured fee receiver:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissioned/ix_update_penalty_fee_receiver.rs#L29

```
staking_state.config.fee_receiver = new_fee_receiver;
```

When `claim_penalty_fee` is called, the program checks that the destination token account is owned by the current configured fee receiver:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissionless/ix_claim_penalty_fee.rs#L43

```
constraint = fee_receiver_token_account.owner ==  
↳ staking.load()?.config.fee_receiver @ ReferralError::InvalidFeeReceiver
```

Therefore, any unclaimed penalty fees are paid to the currently configured fee receiver, even if those fees were accrued while a different fee receiver was configured.

Impact

Previously accrued penalty fees can be redirected to a new fee receiver.

Tool Used

Manual Review

Recommendation

Ensure already-accrued penalty fees cannot be redirected when the fee receiver is changed.

Discussion

`codewithgun`

We acknowledge this. It's a design choice.

Issue L-3: claim_penalty_fee can send fees to any token account owned by fee_receiver, not a configured/ATA account [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/issues/31>

Summary

claim_penalty_fee allows penalty fees to be transferred to any token account owned by the configured fee_receiver, instead of requiring a specific configured token account or the canonical ATA.

Vulnerability Detail

The claim_penalty_fee instruction is permissionless:

<https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/lib.rs#L40-45>

```
// Permissionless instructions
pub fn claim_penalty_fee(
    ctx: Context<ClaimPenaltyFeeCtx>,
    params: ClaimPenaltyFeeParams,
) -> Result<()> {
    handle_claim_penalty_fee(ctx, params)
}
```

The configured fee receiver is stored as a pubkey in the staking config:

<https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/states/staking.rs#L153>

```
/// The account that receives the staking penalty fees
pub fee_receiver: Pubkey,
```

When claiming fees, the program only checks that the provided token account is owned by the configured fee_receiver:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissionless/ix_claim_penalty_fee.rs#L43

```
constraint = fee_receiver_token_account.owner ==
↳ staking.load()?.config.fee_receiver @ ReferralError::InvalidFeeReceiver,
```


Because the destination token account itself is not fixed, an attacker can provide any token account that satisfies:

```
fee_receiver_token_account.owner == staking.config.fee_receiver
```

Impact

If `fee_receiver` is a normal wallet, the impact is low because the wallet authority can usually recover tokens from any token account it owns. However, if `fee_receiver` is a PDA or a program-controlled address, an attacker can create or use a non-canonical token account with:

```
token account owner = fee_receiver PDA
```

Then the attacker can call `claim_penalty_fee` and route the penalty fees into that arbitrary token account.

If the PDA's owning program only knows how to manage a specific ATA or configured token account, it may not be able to recover tokens from the attacker-chosen account. This can cause penalty fees to become stuck or inaccessible.

Tool Used

Manual Review

Recommendation

Store and validate the exact fee receiver token account, rather than accepting any token account owned by `fee_receiver`. For example, store the fee receiver token account in config and validate against it:

```
+ require_keys_eq!(  
+   fee_receiver_token_account.key(),  
+   staking_state.config.fee_receiver_token_account,  
+   ReferralError::InvalidFeeReceiver  
+ );
```

Discussion

codewithgun

We've fixed it in <https://github.com/MeteoraAg/referral-staking/pull/13>

Issue L-4: Pool ownership can be spoofed because owner / new_owner do not need to accept ownership [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/issues/32>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

A staking pool can be initialized or transferred to any arbitrary owner / new_owner pubkey without that account signing or accepting ownership. This can allow an attacker to make a pool appear owned by a trusted protocol wallet that did not consent.

Vulnerability Detail

During initialization, the unchecked owner is stored as the staking owner:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissionless/ix_initialize_staking.rs#L58-59

```
/// CHECK: Owner of the staking, can be any arbitrary account
pub owner: UncheckedAccount<'info>,
```

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissionless/ix_initialize_staking.rs#L86-92

```
staking_state.initialize(
    owner,
    token_vault,
    token_mint,
    staking_penalty_fee_receiver,
    params,
);
```

During owner transfer operation, the new owner is also not checked:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissioned/ix_transfer_owner.rs#L16-17

```
/// CHECK: New owner can be any account, no need to check
pub new_owner: UncheckedAccount<'info>,
```

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissioned/ix_transfer_owner.rs#L25

```
staking_state.owner = ctx.accounts.new_owner.key();
```

Because neither `owner` nor `new_owner` needs to sign, an attacker can create or transfer a pool to a reputable pubkey without that pubkey's consent.

Impact

Users, frontends, or indexers may treat `staking.owner` as a trust signal and believe the pool is operated by a trusted protocol. An attacker can spoof ownership, configure their own fee receiver, and trick users into staking into an unauthentic pool.

Tool Used

Manual Review

Recommendation

Require the initial owner to sign:

```
+pub owner: Signer<'info>,
```

Same fix applies to the ownership transfer operation.

Discussion

codewithgun

We acknowledge this. The program is designed mainly for ourselves to use. Requiring owner to sign create tedious on our setup operation. We will announce the official staking account we use.

Issue L-5: Missing two-step ownership transfer [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/issues/34>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The ownership transfer flow is single-step and does not require the proposed `new_owner` to accept ownership. The current owner can immediately assign ownership to any arbitrary account, including an unwilling account, an incorrect address, or an account that cannot sign.

Vulnerability Detail

The `new_owner` account is unchecked and does not need to sign:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissioned/ix_transfer_owner.rs#L16-17

```
/// CHECK: New owner can be any account, no need to check
pub new_owner: UncheckedAccount<'info>,
```

The only validation is that `new_owner` is different from the current owner. Ownership is then updated immediately:

https://github.com/sherlock-audit/2026-04-meteora-may-4th-2026/blob/main/referral-staking/programs/referral/src/instructions/permissioned/ix_transfer_owner.rs#L22

```
staking_state.owner = ctx.accounts.new_owner.key();
```

Impact

Ownership can be assigned to an account that did not consent to becoming the staking owner. This is not an unauthorized takeover, because the current owner must initiate the operation. The issue is the lack of recipient acceptance and safety around ownership transfer.

Tool Used

Manual Review

Recommendation

Implement a two-step ownership transfer flow.

First, allow the current owner to nominate a pending owner:

```
+staking_state.pending_owner = ctx.accounts.new_owner.key();
```

Then add a separate `accept_owner` instruction where the pending owner must sign:

```
+require_keys_eq!(  
+   staking_state.pending_owner,  
+   ctx.accounts.new_owner.key(),  
+ReferralError::Unauthorized  
+ );  
  
+ staking_state.owner = ctx.accounts.new_owner.key();  
+ staking_state.pending_owner = Pubkey::default();
```

The accepting account should be a signer:

```
+pub new_owner: Signer<'info>,
```

At minimum, require `new_owner` to sign in the existing transfer instruction.

Discussion

codewithgun

We acknowledge this. The program is designed mainly for ourselves to use. Two-step ownership transfer create tedious on our operation side.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.